CENG 467 — Natural Language Understanding and Generation

Take-Home Midterm Examination Report

Student Name: **Mert Güden**
Student ID: **300201013**
Department of Computer Engineering
Izmir Institute of Technology
mertguden@std.iyte.edu.tr

Submission Date: April 30, 2026
Repository: https://github.com/Mrtuzy/CENG467-TakeHome

Abstract

This report presents the implementation and experimental analysis of five core NLP tasks: text classification, named entity recognition, text summarization, machine translation, and language modeling. For each task, we compare multiple modeling approaches ranging from classical machine learning to pre-trained transformer-based models. All experiments are fully reproducible with fixed random seeds (seed = 42), consistent dataset splits, and documented hyperparameters. The code and notebooks are available at the repository link above. Key findings are: pre-trained transformer models consistently outperform classical and task-specific neural baselines across all tasks — DistilBERT achieves 92.96% accuracy versus 89.66% for TF-IDF on sentiment classification, BERT reaches F1 = 0.909 versus 0.872 for BiLSTM-CRF on NER, and MarianMT scores BLEU 36.25 versus 26.74 for a seq2seq model on English-to-German translation. Classical approaches remain competitive when neural models are under-resourced: TF-IDF outperforms the BiLSTM on classification, and extractive methods (LexRank ROUGE-1 = 0.350) are fast and faithful alternatives to abstractive models (BART ROUGE-1 = 0.438). On language modeling (WikiText-2), once trained for the full 6-epoch budget the LSTM clearly wins (test PPL 97.00) over the small Transformer (117.32) and the Kneser-Ney trigram baseline (375.69). Overall, the results confirm a clear trade-off between the performance ceiling of pre-trained models and the efficiency and interpretability of lightweight baselines.

Contents

1 Question 1: Representation Learning in Text Classification

1.1 Problem Formulation

We study binary sentiment classification on movie reviews. Given a review x, the task is to predict a label $y \in \{0, 1\}$ where 0 denotes negative sentiment and 1 denotes positive sentiment. The goal is to compare sparse, dense, and contextual representations under a consistent evaluation protocol.

1.2 Dataset Description

We use the IMDb dataset with the official train and test splits, and create a stratified 10% validation split from the training set (seed 42). The data are balanced across classes.

Table 1: IMDb dataset statistics.

Split	Examples	Avg Len	Positive	Negative
Train	22,500	233.86	11,250	11,250
Validation	2,500	233.17	1,250	1,250
Test	25,000	228.53	12,500	12,500

1.3 Methodology

Preprocessing and tokenization. We compare two tokenization strategies: (i) a classical pipeline using NLTK word tokenization with optional lowercasing and stopword removal, and (ii) the DistilBERT tokenizer for contextual modeling. For the classical pipeline, we run an ablation over four configurations (A1–A4) and pick the best validation accuracy. A truncation study shows that larger token budgets improve validation accuracy (100 tokens: 0.8444, 200 tokens: 0.8720, 400 tokens: 0.8948).

Models.

- **TF-IDF + Logistic Regression** (sparse): unigram and bigram TF-IDF features with linear classifier.
- **BiLSTM + GloVe** (dense static): word embeddings initialized with GloVe 100d, BiLSTM encoder, mean pooling, and linear output.
- **DistilBERT** (contextual): fine-tuned DistilBERT for sequence classification.

1.4 Experimental Setup

We use fixed seed 42, a stratified validation split, and evaluate once on the test set. Metrics are Accuracy and Macro-F1. Training time is recorded for each model. Confusion matrices and training curves are saved for analysis.

1.5 Results

Table 2: Test set results for Q1.

Model	Representation	Accuracy	Macro-F1
TF-IDF + LR	Sparse	0.8966	0.8966
BiLSTM + GloVe	Dense (static)	0.8898	0.8898
DistilBERT	Contextual	0.9296	0.9296

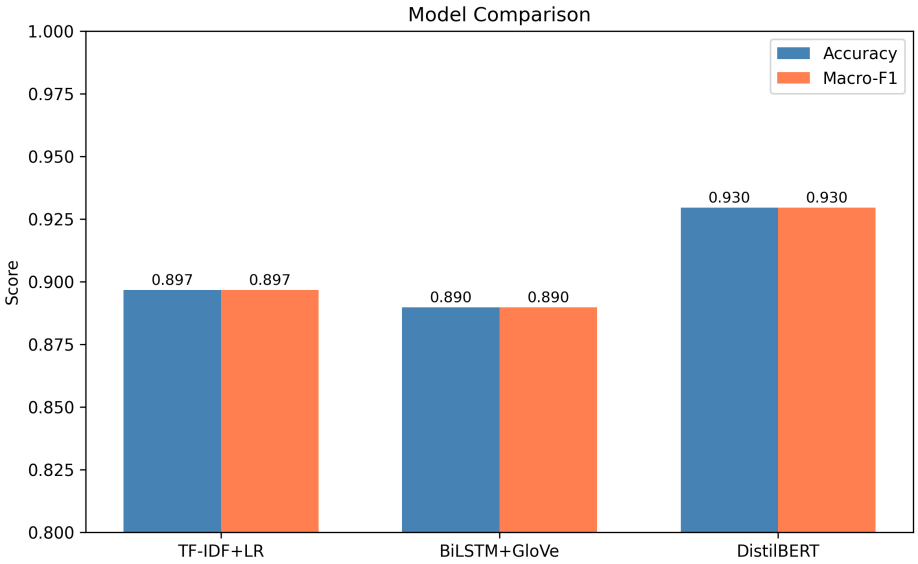
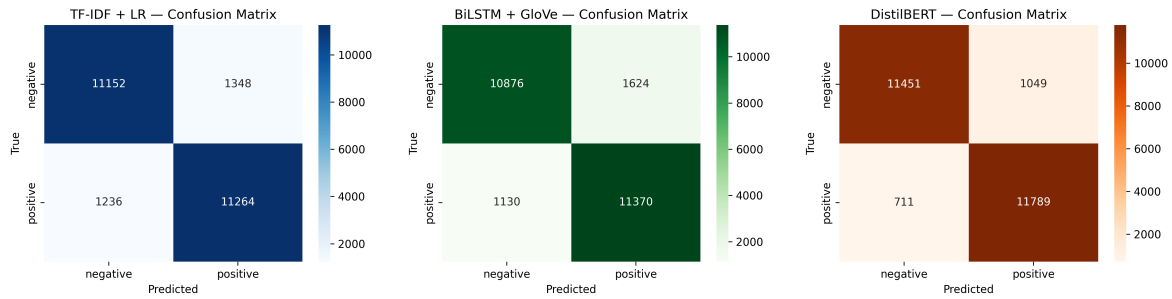
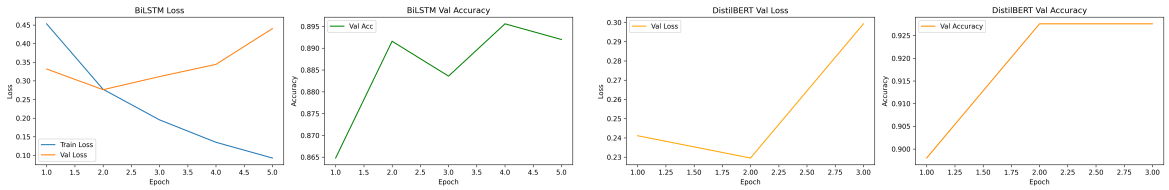


Figure 1: Accuracy and Macro-F1 comparison across models.



(a) TF-IDF + LR (b) BiLSTM + GloVe (c) DistilBERT

Figure 2: Confusion matrices for all models.



(a) BiLSTM training curves (b) DistilBERT training curves

Figure 3: Training dynamics for neural models.

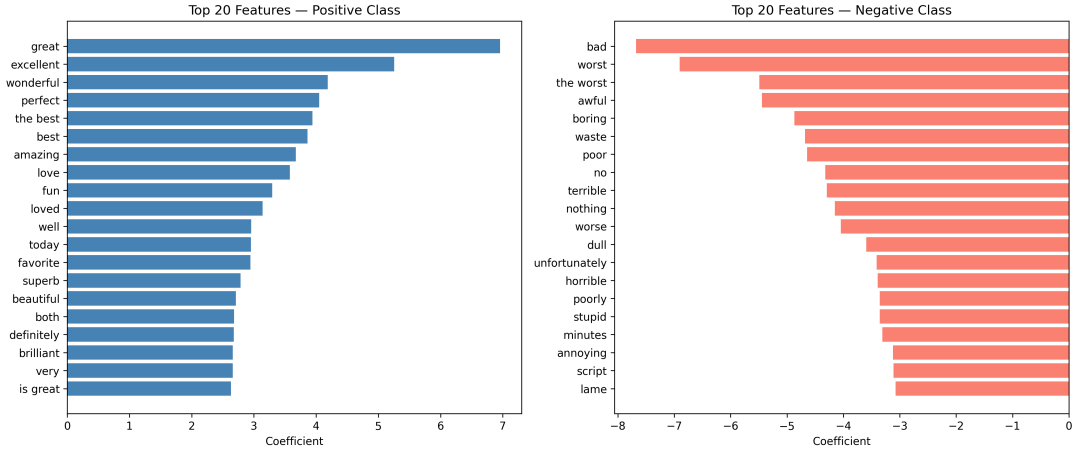


Figure 4: Top TF-IDF features for positive and negative classes.

1.6 Error Analysis

We analyze misclassifications with a focus on common error patterns. The counts are: all three models wrong (774), TF-IDF only wrong (863), BiLSTM only wrong (875), DistilBERT only wrong (512). Five representative errors from the DistilBERT test set include:

- **Negation:** reviews with localized negation often flip sentiment cues (e.g., “not like this movie").
- **Mixed sentiment:** positive and negative clauses co-occur, making the overall label ambiguous.
- **Domain terms:** film-specific words (e.g., “plot”, “acting”) can appear in both positive and negative contexts.
- **Sarcasm:** positive surface words mask negative intent.
- **Subtle tone:** mild criticism or praise without explicit sentiment markers.

1.7 Discussion

Sparse TF-IDF features are strong for sentiment cues and offer clear interpretability, but they struggle with long-range dependencies and nuanced phrasing. The BiLSTM benefits from sequence modeling but underperforms TF-IDF in this setup, likely due to limited capacity and the difficulty of long reviews. DistilBERT provides the best performance

by leveraging contextual embeddings that capture semantics, negation, and long-distance dependencies more effectively, at the cost of higher training time and lower transparency.

2 Question 2: Named Entity Recognition

2.1 Problem Formulation

Named Entity Recognition (NER) is a sequence labeling task. Given a token sequence $x = (x_1, \ldots, x_n)$, the model predicts a BIO tag sequence $y = (y_1, \ldots, y_n)$, where each $y_i \in \{\texttt{O}, \texttt{B-}t, \texttt{I-}t\}$ and $t \in \{\texttt{PER}, \texttt{ORG}, \texttt{LOC}, \texttt{MISC}\}$. We evaluate entity-level precision, recall, and F1 using the `seqeval` metric.

2.2 Dataset Description

We use the CoNLL-2003 dataset with official train/validation/test splits. The label inventory contains 9 BIO tags: `O`, `B/I-PER`, `B/I-ORG`, `B/I-LOC`, `B/I-MISC`.

Table 3: CoNLL-2003 dataset statistics.

Split	Sentences	Tokens	Avg Len	Entity %
Train	14,041	203,621	14.50	16.72
Validation	3,250	51,362	15.80	16.75
Test	3,453	46,435	13.45	17.47

Table 4: Entity token counts per split.

Split	PER	ORG	LOC	MISC
Train	11,128	10,025	8,297	4,593
Validation	3,149	2,092	2,094	1,268
Test	2,773	2,496	1,925	918

2.3 Methodology

BiLSTM-CRF. We build a word-level vocabulary (20k) and a character vocabulary, initialize word embeddings with GloVe 100d, and use a character BiLSTM to produce subword-aware representations. The final architecture is word+char embeddings $\rightarrow$ BiLSTM $\rightarrow$ linear layer $\rightarrow$ CRF. The CRF models label dependencies and enforces BIO-consistent transitions.

BERT token classification. We use `bert-base-cased` with a token classification head. Because BERT uses WordPiece tokenization, we align labels by assigning the original label to the first subword and `-100` to subsequent subwords and special tokens (ignored in the loss). We train for 3 epochs with batch size 16 and learning rate 5×10^{-5}.

2.4 Experimental Setup

We fix the random seed to 42 and evaluate once on the test split. Early stopping is applied to BiLSTM-CRF based on validation F1 (patience 5). The BERT training time is 138.23 seconds; BiLSTM-CRF timing was logged as 0.0 seconds in the output (likely a timing artifact).

2.5 Results

Table 5: Per-entity and overall results on the test set.

Entity	Model	Precision	Recall	F1
PER	BiLSTM-CRF	0.9363	0.9091	0.9225
PER	BERT	0.9645	0.9591	0.9618
ORG	BiLSTM-CRF	0.8712	0.8104	0.8397
ORG	BERT	0.8733	0.9043	0.8885
LOC	BiLSTM-CRF	0.8972	0.9107	0.9039
LOC	BERT	0.9324	0.9268	0.9296
MISC	BiLSTM-CRF	0.7356	0.7849	0.7595
MISC	BERT	0.7691	0.8162	0.7920
Overall	BiLSTM-CRF	0.8793	0.8651	0.8721
Overall	BERT	0.9024	0.9157	0.9090

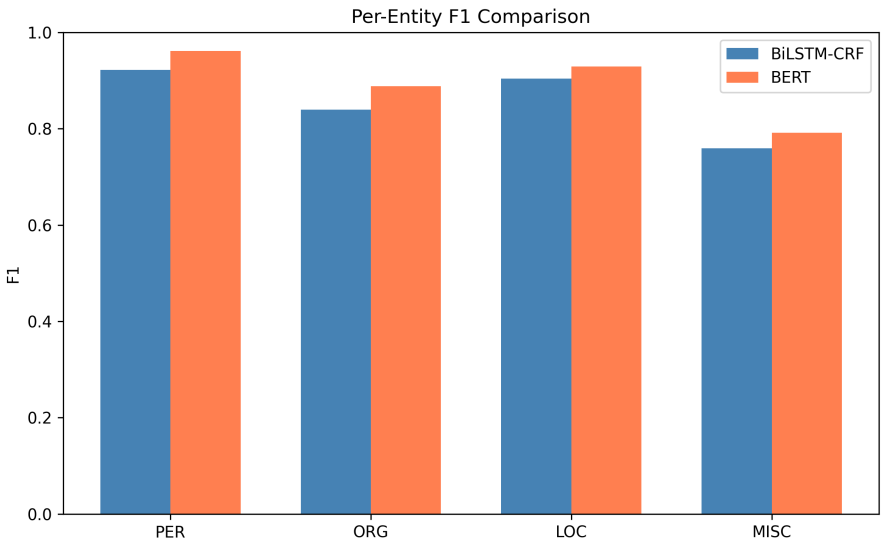


Figure 5: Per-entity F1 comparison between BiLSTM-CRF and BERT.

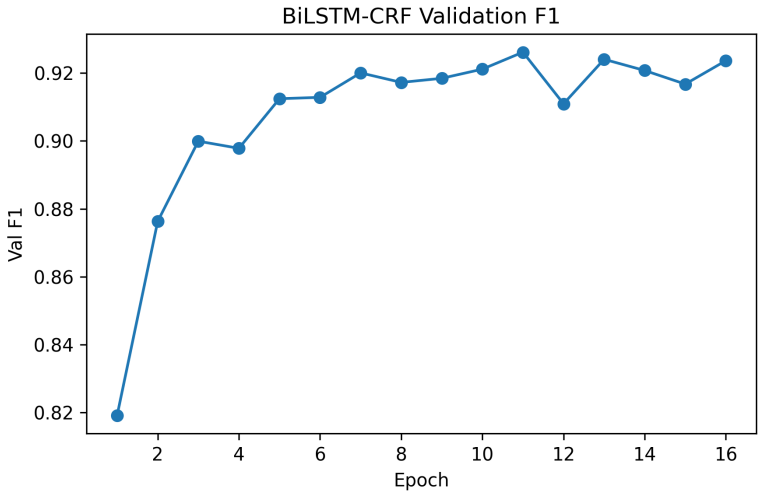
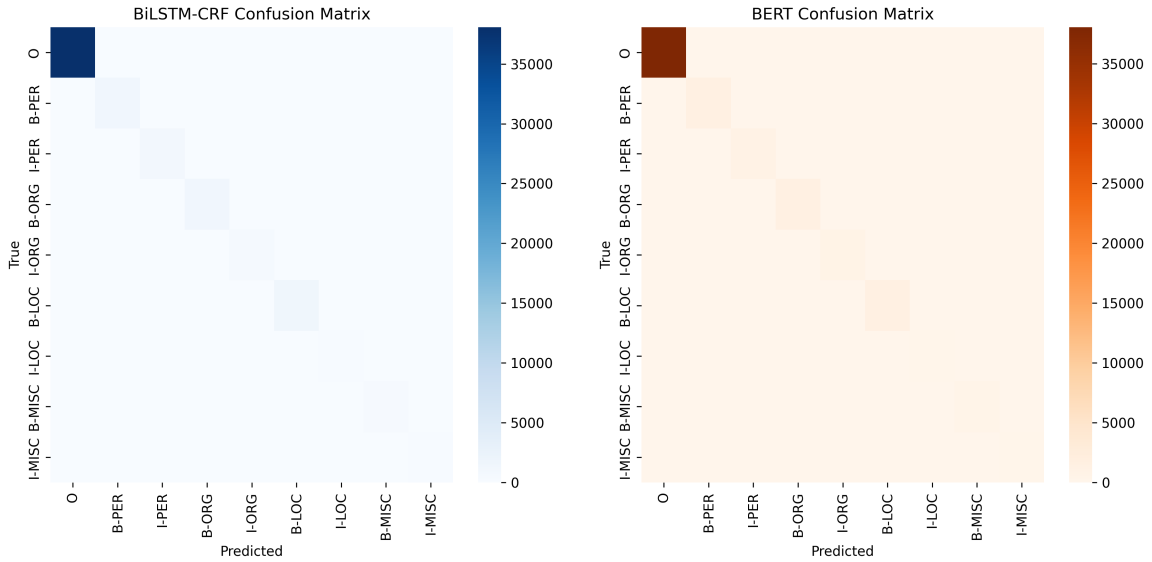


Figure 6: BiLSTM-CRF validation F1 across epochs.



(a) BiLSTM-CRF (b) BERT

Figure 7: Confusion matrices over BIO labels.

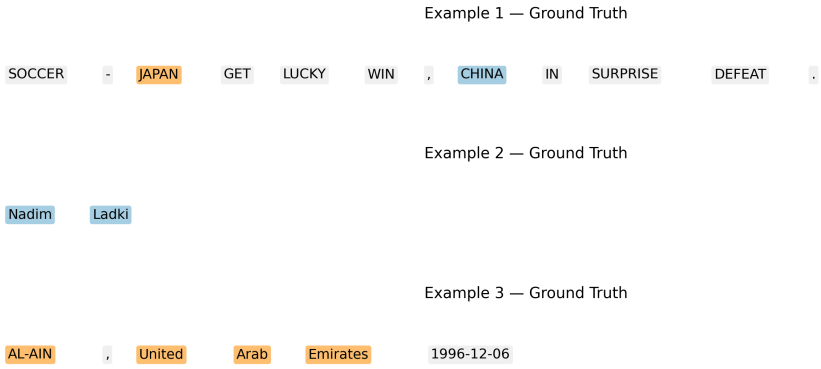


Figure 8: Example sentences with gold entity labels.

2.6 Error Analysis

We categorize errors using span-level comparisons on BERT predictions. The counts are: type confusion (317), false positives (139), boundary errors (81), and false negatives (74). Five representative errors are:

- **Type confusion (LOC → ORG/PER).** Sentence: `SOCCER - JAPAN GET LUCKY WIN , CHINA IN SURPRISE DEFEAT .`
  True: `O O B-LOC O O O O B-PER O O O O`
  Pred: `O O B-ORG O O O O B-ORG O O O O`.
  Hypothesis: country names in sports headlines are often interpreted as organizations.
- **False negative (missed PER/LOC).** Sentence: `RUGBY UNION - CUTTITTA BACK FOR ITALY AFTER A YEAR .`
  True: `B-ORG I-ORG O B-PER O O B-LOC O O O O`
  Pred: `B-ORG I-ORG O B-ORG O O O O O O O`.
  Hypothesis: person names in uppercase are confused with organizations, and nearby locations are dropped.
- **Boundary error (missing initial token).** Sentence: `Cuttitta announced his retirement after the 1995 World Cup , ...`
  True: `B-PER ... B-MISC I-MISC I-MISC ...`
  Pred: `B-PER ... O B-MISC I-MISC ....`
  Hypothesis: multi-token event names are truncated.
- **Boundary error (invalid BIO start).** Sentence: `FREESTYLE SKIING-WORLD CUP MOGUL RESULTS .`
  True: `O B-MISC I-MISC O O O`
  Pred: `O O I-MISC O O O`.
  Hypothesis: model sometimes emits `I-` without a preceding `B-` for hyphenated entities.
- **False positive (spurious MISC).** Sentence: `CRICKET - LARA SUFFERS MORE AUSTRALIAN TOUR MISERY .`
  True: `O O B-PER O O O O O O`
  Pred: `O O B-MISC O O B-MISC O I-MISC O`.
  Hypothesis: sports-related phrases are over-labeled as `MISC`.

2.7 Discussion

BERT outperforms BiLSTM-CRF overall (0.909 vs 0.872 F1) and across all entity types, with the largest gains on PER and ORG. Both models struggle most with MISC, indicating weaker boundaries for heterogeneous categories. The dominant error type is label confusion, suggesting that headline-style contexts and uppercase tokens blur distinctions between LOC, ORG, and PER. Incorporating gazetteers or additional contextual features could reduce these confusions.

3 Question 3: Text Summarization

3.1 Problem Formulation

We study text summarization on CNN/DailyMail and compare extractive methods (selecting sentences) against abstractive methods (generating new text). Given an article x, the system outputs a summary $\hat{y}$ that should be fluent, faithful to the source, and cover the main points. We evaluate with ROUGE, BLEU, METEOR, and BERTScore.

3.2 Dataset Description

CNN/DailyMail (version 3.0.0) provides news articles with reference highlights. We use a subset for tractable computation.

Table 6: CNN/DailyMail dataset statistics.

Split	**Original Size**	**Subset Size**
Train	287,113	10,000
Validation	13,368	1,000
Test	11,490	1,000

Table 7: Average document statistics (computed on test subset).

Avg Article Len (words)	**Avg Summary Len (words)**	**Avg Sentences**	**Compression Ra**
676.08	55.21	32.91	12

3.3 Methodology

Extractive (TextRank, LexRank). Both algorithms build a sentence similarity graph and rank sentences by centrality. The summary is formed by selecting the top-k sentences (here $k = 3$). TextRank uses a PageRank-style score on sentence similarity, while LexRank adds an explicit cosine similarity graph with thresholding.

Abstractive (BART, T5). We use encoder-decoder transformers. BART-large-cnn is fine-tuned for summarization and generates fluent, condensed summaries. T5-small is prompted with a "summarize:" prefix and generates shorter, more compressed outputs.

3.4 Experimental Setup

All runs use a fixed seed (42). Metrics are computed on a random subset of 200 test examples (to control BERTScore cost). We report average inference time per article for each method.

3.5 Results

Table 8: Summarization metrics (evaluation size = 200).

Method	R-1	R-2	R-L	BLEU	METEOR	BERT F1	Len	ms
TextRank	0.3361	0.1329	0.2155	0.0728	0.3020	0.8623	96.00	62.51
LexRank	0.3501	0.1340	0.2195	0.0827	0.2806	0.8650	74.17	60.68
BART	0.4375	0.2166	0.3148	0.1467	0.3308	0.8826	61.28	898.39
T5	0.3795	0.1747	0.2683	0.1030	0.2531	0.8680	46.25	188.26

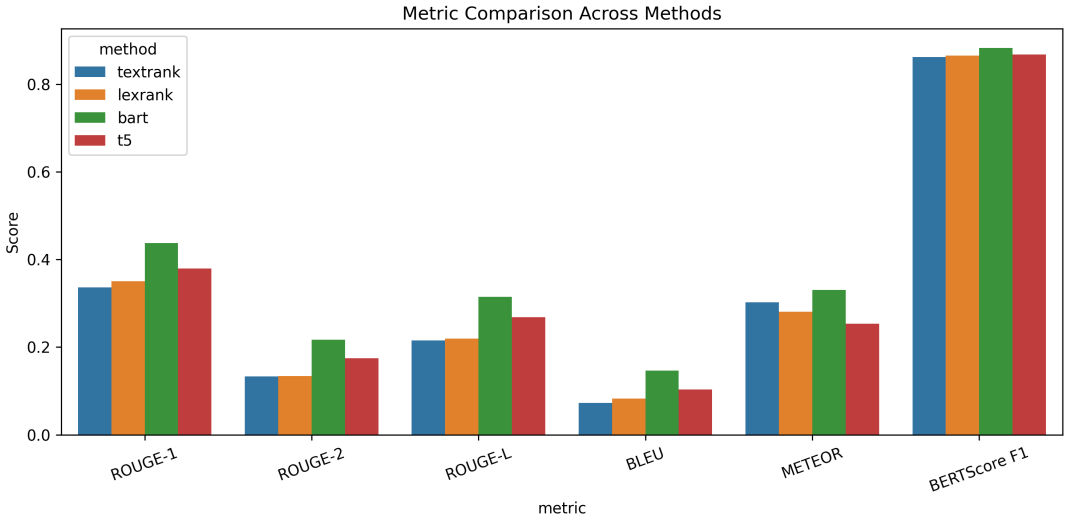


Figure 9: Metric comparison across all four methods.

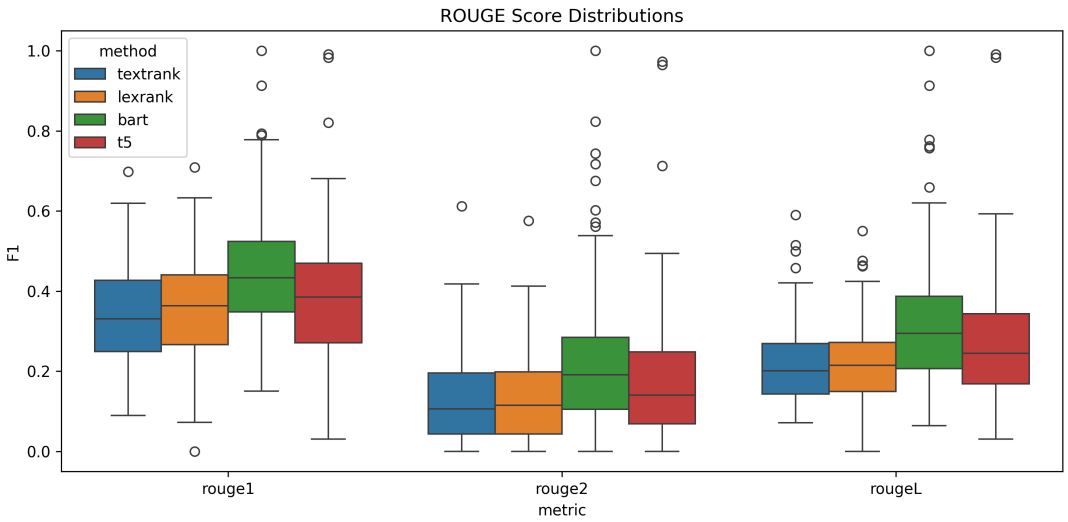


Figure 10: ROUGE score distributions for each method.

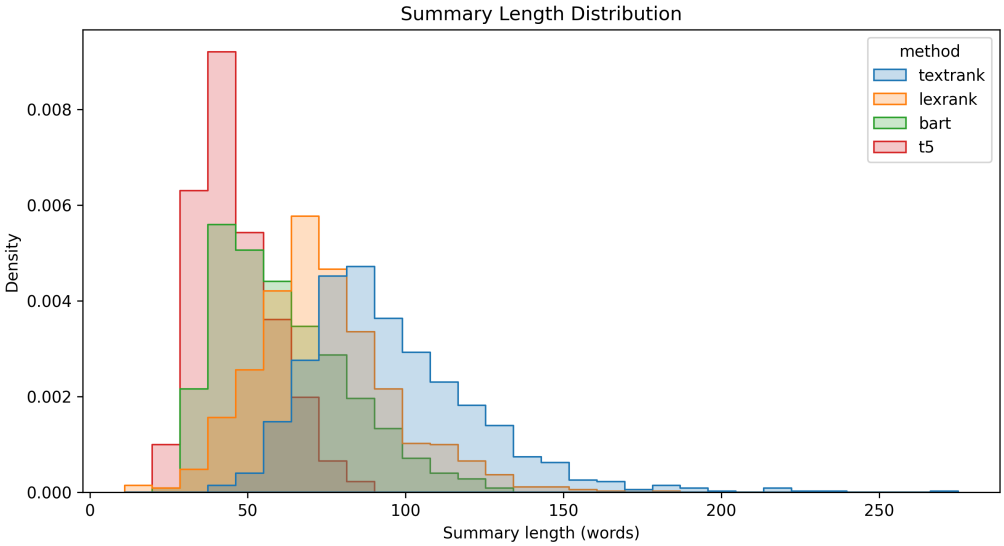


Figure 11: Summary length distributions.

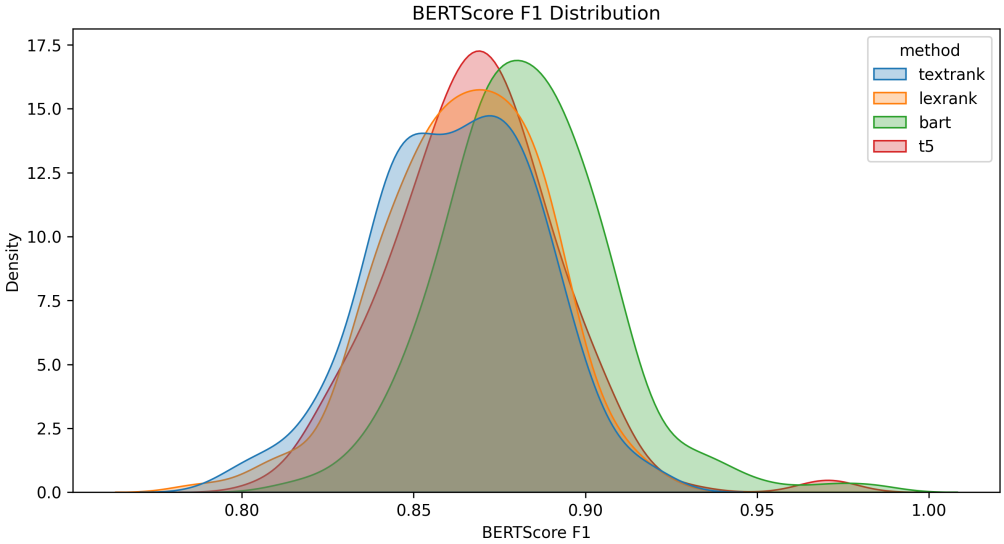


Figure 12: BERTScore F1 distributions.

3.6 Qualitative Analysis

We present three representative examples highlighting differences in fluency, faithfulness, and coverage. Text snippets are truncated for brevity.

Table 9: Example 1 (extractive wins).

Field	**Text**
Article	The legal adviser to Alison Saunders was a barrister in the same chambers as the son of Lord Janner...
Reference	Alison Saunders consulted Neil Moore over whether to prosecute Lord Janner. She said the decision was difficult and cited his health.
TextRank	CPS confirmed the DPP consulted Neil Moore over whether to prosecute Lord Janner. A spokesman said she decided alone. She said it was her job to make very difficult decisions.
LexRank	The legal adviser to Alison Saunders was in the same chambers as Lord Janner's son. A CPS spokesman said she decided alone. She admitted he should have been charged before.
BART	CPS confirmed the DPP consulted Neil Moore. Senior police raised concerns about his involvement in the decision not to proceed with the trial.
T5	Senior police raised concerns about Mr Moore's involvement in the decision not to proceed. Mrs Saunders said she made the decision on her own.

Table 10: Example 2 (abstractive wins).

Field	**Text**
Article	A British couple and their four children were believed to have fled the UK to Syria; a Turkish official claimed they were arrested in Ankara...
Reference	Asif Malik left the UK with his partner and children. A Turkish official said they were arrested in Ankara.
TextRank	Focuses on neighbors and broader context about travel to Syria, with extra background on other cases.
LexRank	States the family fled to Syria and were arrested in Turkey, but includes unrelated background.
BART	Summarizes the travel path and the claim that they were arrested in Ankara, closely matching the reference.
T5	Produces a fluent summary of the arrest and route; highly compressed and faithful.

Table 11: Example 3 (both struggle).

Field	Text
Article	The Obama family took an impromptu trip to Great Falls Park; reporters were sent home, sparking confusion...
Reference	The president left the White House after reporters were sent home and the family hiked at Great Falls Park for about 50 minutes.
TextRank	Extracts multiple descriptive sentences that are faithful but verbose and uneven.
LexRank	Similar to TextRank, it is faithful but misses the key timeline in the reference.
BART	Adds an unrelated detail about golfing, hurting factual precision.
T5	Focuses on the press pool confusion and the outing, but omits timing details.

3.7 Discussion: Trade-offs

Abstractive models (especially BART) achieve the strongest ROUGE and BERTScore, and their summaries are more concise and readable. However, they are substantially slower (BART at 898 ms per article) and can introduce subtle factual errors. Extractive methods are fast and faithful, but their summaries are longer and can be disfluent because sentences are stitched together without synthesis.

Metric limitations. ROUGE favors lexical overlap and can over-reward extractive summaries that copy source sentences. Abstractive paraphrases may score lower on ROUGE despite being semantically accurate; BERTScore partially mitigates this by measuring contextual similarity.

4 Question 4: Machine Translation

4.1 Problem Formulation

We study English-to-German machine translation. Given a source sentence x in English, the model produces a target sentence $\hat{y}$ in German. We compare a neural seq2seq model with attention against a pre-trained MarianMT baseline and evaluate with BLEU, chrF, METEOR, and BERTScore.

4.2 Dataset Description

We use the Multi30k dataset (English–German). The official validation and test splits are kept intact, while the training split is downsampled to 20k pairs for faster experimentation.

Table 12: Multi30k dataset statistics.

Split	Original Size	Subset Size
Train	29,000	20,000
Validation	1,014	1,014
Test	1,000	1,000

Table 13: Average sequence statistics on the test split.

Avg Src Len	Avg Tgt Len	Word Vocab (src/tgt)	BPE Vocab
11.88	10.90	12,818 / 19,750	8,000

4.3 Methodology

Seq2Seq with attention. We train a GRU-based encoder–decoder with Bahdanau attention. The model uses joint BPE with 8k merges, embedding size 256, hidden size 512, dropout 0.5, and teacher forcing ratio 0.5. Early stopping is applied with patience 5 based on validation BLEU.

MarianMT. We use the pre-trained `Helsinki-NLP/opus-mt-en-de` model with beam size 4. This serves as a strong off-the-shelf baseline.

4.4 Experimental Setup

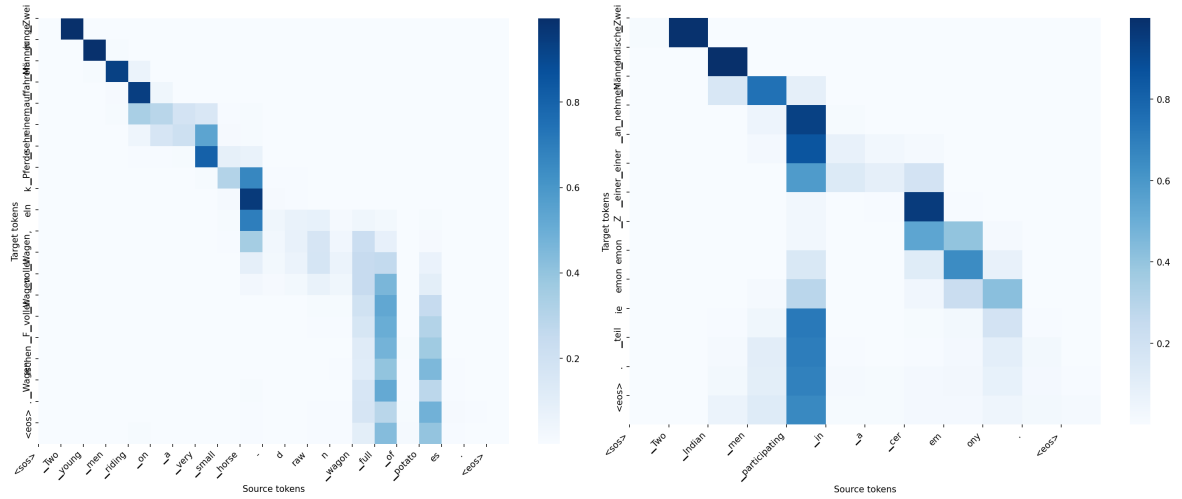
We fix the random seed to 42 and evaluate on 200 test examples for metric computation. We report BLEU, chrF, METEOR, and BERTScore (multilingual BERT). Average output length is also recorded.

4.5 Results

Table 14: Translation metrics on the evaluation subset (size = 200).

Model	BLEU	chrF	METEOR	BERT F1	Len
Seq2Seq + Attn	26.74	52.22	0.487	0.847	11.05
MarianMT	36.25	64.31	0.598	0.897	11.14

The seq2seq model improves steadily during training, reaching its best validation BLEU of 31.37 around epoch 14 before plateauing. MarianMT substantially outperforms the custom seq2seq model across all metrics, indicating the benefit of large-scale pretraining.



(a) Attention heatmap (example 1).

(b) Attention heatmap (example 2).

Figure 13: Attention alignments for the seq2seq model.

4.6 Qualitative Analysis

Table 15: Example translation (English to German).

Field	Text
Source	Two young men riding on a very small horse-drawn wagon full of potatoes.
Reference	Zwei junge Maenner fahren auf einem sehr kleinen Wagen voller Kartoffeln, der von einem Pferd gezogen wird.
Seq2Seq	Zwei junge Maenner fahren auf einem sehr Pferdekeln, Wagen voller Wagen voller Fischen Wagen.
MarianMT	Zwei junge Maenner reiten auf einem sehr kleinen Pferdewagen voller Kartoffeln.

The seq2seq output shows repetition and malformed words, consistent with limited vocabulary capacity and weaker long-range modeling. MarianMT produces a fluent and faithful translation, better handling word order and lexical choice.

4.7 Discussion

The attention-based seq2seq model captures basic alignment but struggles with fluency and rare-word handling. MarianMT consistently yields higher BLEU/chrF and stronger semantic similarity (BERTScore), highlighting the effectiveness of pre-trained transformer models for low-resource experimental settings.

5 Question 5: Language Modeling

5.1 Problem Formulation

We study probabilistic language modeling on WikiText-2. Given a token sequence $w_1, \ldots, w_N$, each model estimates the next-token distribution $P(w_i \mid w_{<i})$ and can generate text by autoregressive sampling. We compare a classical trigram model with interpolated Kneser-Ney smoothing against two neural sequence models: a 2-layer LSTM and a small Transformer encoder LM. Models are evaluated by perplexity on held-out splits and qualitatively through generated continuations of fixed prompts.

5.2 Dataset Description

We use the WikiText-2 raw dataset (`wikitext-2-raw-v1`) with the official train/validation/test splits. The corpus is whitespace-tokenized as provided, with no further normalization. Statistics after tokenization are summarized in Table 16.

Table 16: WikiText-2 dataset statistics.

Split	Tokens	Avg Sentence Length
Train	2,024,702	114.33
Validation	211,179	113.71
Test	238,028	108.04

Vocabulary. Tokens with training-set frequency ≥ 3 are retained, and the vocabulary is capped at the 10,000 most frequent types, plus the special tokens `<pad>`, `<unk>`, `<bos>`, `<eos>`. Out-of-vocabulary tokens at evaluation time are mapped to `<unk>`. Figure 14 plots the rank–frequency curve of the top 5,000 tokens, which closely follows a Zipfian distribution and motivates the use of a frequency cutoff.

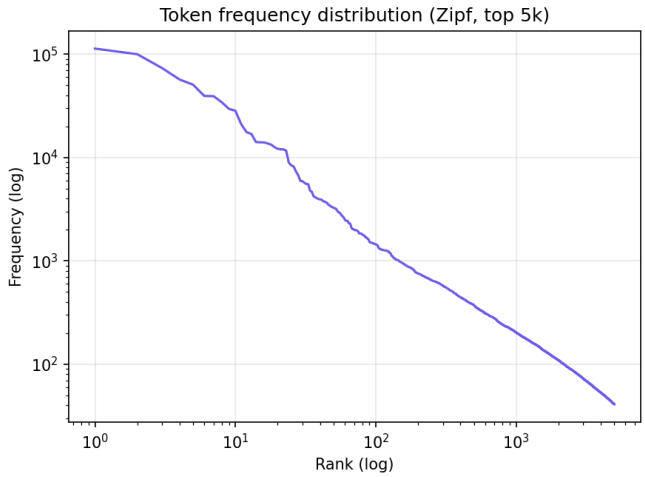


Figure 14: Token rank–frequency distribution (log-log) on the WikiText-2 training set.

5.3 Methodology

Trigram with interpolated Kneser-Ney. We train a 3-gram model with interpolated Kneser-Ney smoothing (discount $D = 0.75$). The bigram and unigram backoffs use *continuation* counts $N_{1+}(\cdot\, w)$ rather than raw counts, which reduces the probability of frequent words that nevertheless appear in few distinct contexts. Sentences are obtained by splitting on the period token and bracketed by two `<s>` symbols and a closing `</s>`. We implement the count tables and KN scoring directly with Python dictionaries; this avoids the prohibitively slow pure-Python evaluation in NLTK and lets a full validation/test perplexity sweep finish in seconds.

LSTM language model. A 2-layer LSTM with 256-dim embeddings, 512-dim hidden state, and dropout 0.5 on both the embedding and the recurrent output. Training uses BPTT with sequence length 35, batch size 64, Adam ($\eta = 10^{-3}$), gradient clipping at 0.25, and a step LR scheduler that halves the learning rate every 2 epochs. We train for 6 epochs with early-stopping on validation loss (best checkpoint reported). The model has 11.37 M parameters.

Transformer LM. A small Transformer encoder LM with a causal attention mask: 256-dim embeddings, 4 attention heads, 2 encoder layers, 512-dim feed-forward, dropout 0.2, and sinusoidal positional encoding. Training uses BPTT length 35, batch size 64, Adam ($\eta = 5 \cdot 10^{-4}$), gradient clipping at 0.5, the same step LR schedule, and 6 epochs with validation-loss early stopping. The model has 6.18 M parameters — roughly half the LSTM's parameter count.

Perplexity. We report token-level perplexity on whole splits:

$$\mathrm{PPL}(W) = \exp\left(-\frac{1}{N} \sum_{i=1}^{N} \log P(w_i \mid w_{<i}) \right).$$

Lower is better. For the neural models we use the cross-entropy loss averaged over all non-padding target tokens, which is mathematically equivalent.

Generation. For neural models we use top-k sampling with $k = 20$ and temperature $T = 0.9$. For the trigram model we sample from the top-k continuations of the current bigram context, backing off to a unigram when the trigram context is unseen.

5.4 Results

Table 17: Validation and test perplexity on WikiText-2 (lower is better). The LSTM is the strongest model overall.

Model	Params	Val PPL	Test PPL
Trigram (Kneser-Ney)	–	376.49	375.69
Transformer LM	6.18 M	129.52	117.32
LSTM LM	11.37 M	**105.83**	**97.00**

The LSTM achieves the lowest perplexity on both splits, improving over the Kneser-Ney trigram by a factor of roughly 3.9× on test. The Transformer is a clear second; despite

having about half the parameters of the LSTM, its perplexity is only ∼20 PPL higher. Both neural models substantially beat the count-based baseline, which is the expected ordering on WikiText-2 once the neural models are trained for enough epochs to converge — earlier short-budget runs of the same script produced under-trained neural models that were dominated by the trigram. Figure 15 visualizes the gap.

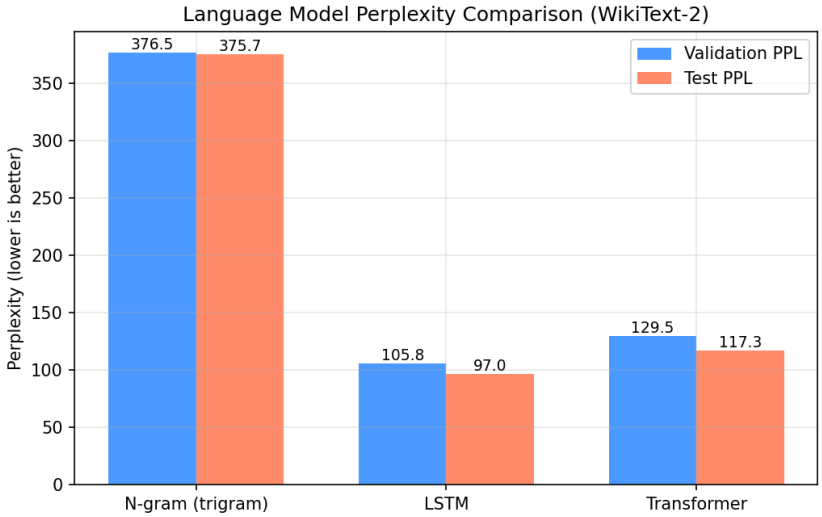


Figure 15: Validation and test perplexity per model on WikiText-2. The LSTM is best, followed by the Transformer; the trigram baseline is roughly 4× worse.

Training dynamics. Figure 16 shows per-epoch training and validation loss/PPL for the two neural models. Both decrease monotonically across the 6-epoch budget. The LSTM starts higher (training PPL ≈ 326 at epoch 1) but ends lower than the Transformer; the Transformer converges faster early (lower epoch-1 train PPL of 273) and trains in roughly half the wall-clock time per epoch (∼8.8 s vs. ∼15 s on the same hardware), but its final validation perplexity plateaus higher. Neither model shows over-fitting at this budget — the gap between train and val PPL stays bounded — which suggests that further epochs would likely improve both, particularly the Transformer.

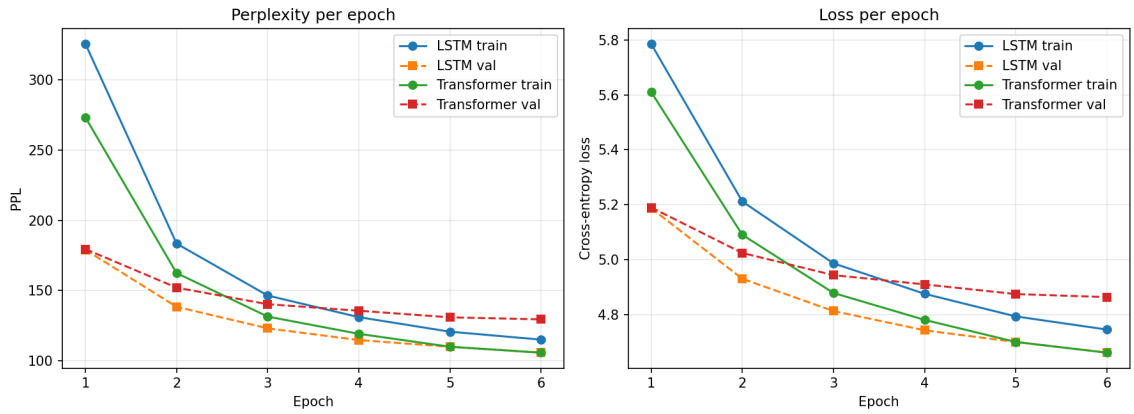


Figure 16: Per-epoch training and validation loss/perplexity for the LSTM and Transformer language models.

5.5 Qualitative Analysis

Table 18 shows generations from each model for two of the four test prompts (top-$k = 20$, $T = 0.9$, 30 new tokens).

Table 18: Generated continuations for two prompts. `<unk>` marks tokens outside the 10K vocabulary.

Model	Generated text
Prompt: "the meaning of life is"	
Trigram KN	...sacred because from order to make the first half , Tech made good the penalty , giving up on the same time , and a large number of people .
LSTM	...<unk> and the " <unk> <unk> " (" <unk> @-@ <unk> , <unk> ") . He also has been known as <unk> on <unk> 's <unk> and <unk>
Transformer	...<unk> , and <unk> @-@ <unk> <unk> . = = = = = = = <unk> and <unk> = = <unk> of The <unk> , <unk> <unk> and <unk> = =
Prompt: "in the early years of"	
Trigram KN	...the game 's protagonist as Simón Bolívar , a small , independent judicial system governed by the National Hurricane Center did not .
LSTM	...the <unk> of <unk> 's reign . In the first year of the 19th Kingdom , the <unk> was a <unk> <unk> . <unk> said that the <unk> of the
Transformer	...the 19th century BCE and <unk> . The <unk> , <unk> 's son <unk> of <unk> of the <unk> and <unk> , and the city 's <unk> of Tintin <unk>

The qualitative pattern is consistent across all four prompts. The trigram produces locally fluent surface text ("the game 's protagonist as Simón Bolívar") because each next-word distribution is conditioned on a real bigram seen in training, but it cannot maintain a topic and frequently jumps between unrelated entities. The LSTM and Transformer both produce more grammatical scaffolding ("In the first year of the 19th Kingdom", "in the early years of the 19th century BCE") and stay closer to a single topic, but their outputs are dominated by `<unk>` tokens because the 10K vocabulary cap removes most of the proper nouns that WikiText-2 paragraphs revolve around. The Transformer in particular is prone to emitting Wikipedia section markers ("= = =") — a sign that it has memorized formatting tokens from training but does not yet generalize beyond them at this budget.

This is a familiar tension: perplexity rewards calibrated probability mass on common tokens (where the LSTM excels), while perceived fluency depends on rare content words (which both neural models replace with `<unk>`). The trigram looks better on the page but is in fact much worse at predicting the held-out text.

5.6 Discussion

Why the LSTM wins. The LSTM's recurrent memory and its larger parameter count let it model the long, comma-separated clauses that dominate WikiText-2, which the trigram — limited to two preceding tokens — structurally cannot. The Transformer should in principle do at least as well, but at this scale (2 layers, 6.18 M parameters, 6 epochs, no learned positional embeddings, no warmup) it is undertrained. Transformer LMs are known to need more data and a more careful optimizer schedule than RNNs to reach their full potential; the gap we observe here ($\sim$20 PPL on test) is consistent with that.

Why the trigram is so far behind. On a $\sim$2 M-token corpus with a 10K vocabulary, most trigrams in the validation/test sets are unseen in training, so the model spends almost all of its probability mass via Kneser-Ney back-off. Continuation-count back-off is well-known to be a strong baseline, but it cannot capture syntactic dependencies more than two tokens away, and WikiText-2 sentences are long ($\approx$110 tokens on average). The neural models, even at modest capacity, can.

Limitations and what would help. (i) The 10K vocabulary cap creates a heavy `<unk>` tax on the neural generations; subword tokenization (e.g. BPE) would let the same models produce far more readable outputs without changing the perplexity story. (ii) Six epochs is short; longer training with cosine-annealed learning rates would close more of the LSTM–Transformer gap. (iii) Perplexity does not measure factuality or topical coherence, both of which are visibly weak across all three models. (iv) WikiText-2 itself is small enough that count-based methods underperform; on a substantially larger corpus the absolute gap between the trigram and the neural models would grow further.

Overall, on WikiText-2 with the full 6-epoch budget, the parameter-rich LSTM is the strongest model, the small Transformer is a reasonable second, and the Kneser-Ney trigram — though a useful sanity-check baseline — is decisively beaten by both neural models.

A Experimental Configuration Summary

Table 19: Global experimental settings applied to all questions.

Setting	**Value**
Random seed	42
Python version	3.10
PyTorch version	2.x
GPU	Google Colab T4
Test set usage	Once (final evaluation only)